master 1 branch 0 tags Go to file Add file Code

JakubNei Better least recently used cache for chunk data 535e47c on Jul 6, 2021 107 commits

Assets	Better least recently used cache for chunk data	2 years ago
Links	no message	6 years ago
Logs	Various Unity generated files	3 years ago
Packages	Updated Packages	2 years ago
ProjectSettings	Updated ProjectSettings	2 years ago
UserSettings	Various Unity generated files	3 years ago
readme_images	Readme images are in this git repository	2 years ago
.gitignore	initial, took me 4 hours yesterday, copied and edited code from diplo...	6 years ago
LICENSE.txt	updated readme	6 years ago
readme.md	Readme images are in this git repository	2 years ago

readme.md

About

This is where I try to refine and improve on everything I've learned in my Procedural planets generator. The idea is that someday this will be license free Earth sized comprehensive Unity GPU-mostly procedural planets and procedural universe generator. So people can #MakeGamesNotProceduralPlanets

It's not production ready !

Key technological points

- A base (planetary) height map is provided or generated, which defines the basic planet shape
- Uses chunked LOD quad tree
- Chunk normal maps are in model space
- Each chunk is located on planet inside area of 4 direction unit vectors

Features

- Floating scene origin
- Procedural space skybox

Chunk generation steps

1. GPU: generate chunk height map from base planetary height map using bicubic sampling
2. GPU: generate chunk slope map from chunk height map
3. GPU: chunk height map: add noise based on chunk slope map
4. GPU: fill array of chunk mesh vertices from chunk height map
5. GPU: generate chunk slope map from chunk height map
6. GPU: generate chunk diffuse map based on chunk slope map
7. GPU: generate chunk normal map based on chunk height map
8. GPU -> CPU: download chunk mesh vertices from GPU to CPU
9. CPU: move edge vertices down to create skirt to hide cracks on different LODs boundary (optional)
10. CPU: create chunk mesh (and mesh collider) from downloaded chunk mesh vertices

Chunk mesh vertices could be generated on CPU only, so it can be used on dedicated servers. Things generated on GPU should be only to add eye candy. Currently everything is generated on GPU.

Todo

- Fix normal map artefacts if planet has radius over 10k.
- Better biomes, per biome compute shader, that outputs: weight, color and adjusts height.
- Noise channels (Outerra), that adds octaves as chunk depth increases, so we won't need to calculate 30 octaves every time.

Planet detail subdivision (chunked LOD quad tree)

You can use either squares or triangles for chunks shape. Triangles appear to have better mesh, but it's finicky to figure out their texturing coordinates, plus you use only half of texture for each tringular chunk. Root triangular chunks are made out of icosahedron. Root square chunks are made out of cube.

Naive cube to sphere function is:

```
unitSphere = normalize(unitCube);
```

this however brings in distortions, better distortion-less unit cube to unit sphere is:

```
// -1 <= unitCube.x && unitCube.x <= 1
// -1 <= unitCube.y && unitCube.y <= 1
// -1 <= unitCube.z && unitCube.z <= 1
// uses math from http://mathproofs.blogspot.cz/2005/07/mapping-cube-to-sphere.html
// implementation license: public domain
float3 unitCubeToUnitSphere(float3 unitCube)
{
        float3 unitCubePow2 = unitCube * unitCube;
        float3 unitCubePow2Div2 = unitCubePow2 / 2;
        float3 unitCubePow2Div2x3 = unitCubePow2 / 3;
        return unitCube * sqrt(1 - unitCubePow2Div2.yzx - unitCubePow2Div2.zyx + unitCubePow2.yzx * unitCubeP
}
```

```
/// <summary>
/// transforms unitCube position into unitSphere,
/// implementation license: public domain,
/// uses math from http://mathproofs.blogspot.cz/2005/07/mapping-cube-to-sphere.html
/// </summary>
/// <param name="unitCube">unitCube.xyz is in inclusive range [-1, 1]</param>
/// <returns></returns>
public static Vector3 UnitCubeToUnitSphere(Vector3 unitCube)
{
        var unitCubePow2 = new Vector3(unitCube.x * unitCube.x, unitCube.y * unitCube.y, unitCube.z * unitCub
        var unitCubePow2Div2 = unitCubePow2 / 2;
        var unitCubePow2Div3 = unitCubePow2 / 3;
        var unitSphere = new Vector3(
                unitCube.x * Mathf.Sqrt(1 - unitCubePow2Div2.y - unitCubePow2Div2.z + unitCubePow2.y * unitCu
                unitCube.y * Mathf.Sqrt(1 - unitCubePow2Div2.z - unitCubePow2Div2.x + unitCubePow2.z * unitCu
                unitCube.z * Mathf.Sqrt(1 - unitCubePow2Div2.x - unitCubePow2Div2.y + unitCubePow2.x * unitCu
        );
        return unitSphere;
}
```

Chunk height maps

Chunk height maps can have adjusted min max, based on the range they need. That is what the FindTextureMinMax is for.

```
float chunkMapHeight = (realPlanetHeight - _heightMin) / (_heightMax - _heightMin);

float realPlanetHeight = chunkMapHeight * (_heightMax - _heightMin) + _heightMin;
```

Child chunk height map is based off of parent chunk height map

Links

Normal maps blending aproaches

fBM, Billowy turbulence, Ridged turbulence, IQ Noise

Introduction series to making procedural worlds

Outerra: Bicubic sampling, Slope depedent noise example

Screenshot: Chunk skirts in Elite Dangerous

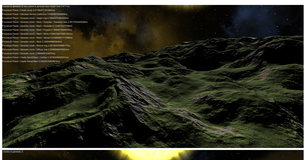
GPU-Based Procedural Terrains in 'Serious Sam 4: Planet Badass'

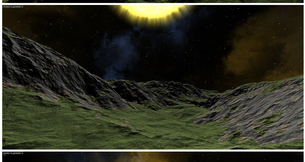
← Sean Murray GDC Talk (2017) | Building Worlds with Noise Generation | No Man's Sky | [HQ]

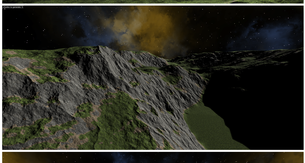
Coding Adventure: Procedural Moons and Planets

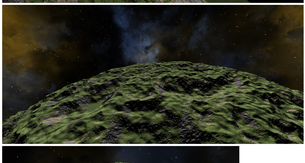
Star Citizen Live: Making Mountains

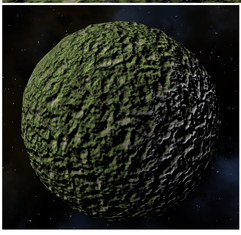
Screenshots



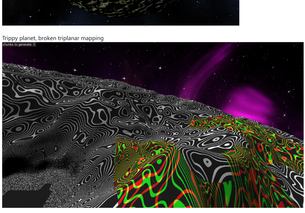




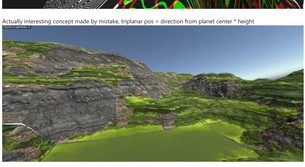




Trippy planet, broken triplanar mapping.



Actually interesting concept made by mistake, triplanar pos = direction from planet center * height



© 2023 GitHub, Inc. Terms Privacy Security Status Docs Contact GitHub Pricing API Training Blog About

About
Unity Procedural Planets GPU-mostly generator
Readme
Unlicense license
114 stars
22 watching
22 forks
Report repository

Releases
No releases published

Packages
No packages published

Contributors 2
JakubNei Jakub Nei
netri Netri

Languages
C# 50.6% HLSL 44.8%
ShaderLab 4.7%